

FASCICOLO ASSEMBLER PER RISC-V (istruzioni tratte da ISA RV64i)

I registri *rd*, *rs1* e *rs2* elencati nelle (pseudo-)istruzioni indicano registri a 64 bit da *x0* a *x31*.

CONTROLLO – pseudo-istruzioni

<i>nop</i>			istruzione <i>nulla</i>
------------	--	--	-------------------------

ARITMETICA – istruzioni native (operazioni in aritmetica in complemento a 2) – vedi nota 1

<i>add</i>	<i>rd</i> , <i>rs1</i> , <i>rs2</i>	$rd := rs1 + rs2$	<i>addizione</i> a 64 bit
<i>addi</i>	<i>rd</i> , <i>rs1</i> , <i>cost12</i>	$rd := rs1 + cost12$	<i>addizione</i> a 64 bit di <i>costante</i>
<i>sub</i>	<i>rd</i> , <i>rs1</i> , <i>rs2</i>	$rd := rs1 - rs2$	<i>sottrazione</i> a 64 bit

L'istruzione *oddi* prima estende in segno la costante *cost12* da 12 bit fino a 64 bit, poi la somma al registro sorgente *rs1* e infine scrive il risultato nel registro destinazione *rd*.

ARITMETICA – pseudo-istruzioni (operazioni in aritmetica in complemento a 2)

<i>neg</i>	<i>rd</i> , <i>rs1</i>	$rd := -rs1$	<i>negazione</i> aritmetica (inversione di segno in compl. a 2)
------------	------------------------	--------------	---

Nota 1: esistono anche le istruzioni macchina native di moltiplicazione (*mul*) e divisione intera, con calcolo separato di quoziente (*div*) e resto (*rem*), in aritmetica in complemento a 2 con operandi e risultato da 64 bit; ecco le istruzioni base: *mul / div / rem rd, rs1, rs2*; queste istruzioni calcolano $rd := rs1 \times rs2 / rs1 \div rs2 / rs1 \bmod rs2$, rispettivamente; appartengono allo ISA esteso RV64im e hanno numerose varianti.

CONFRONTO – istruzioni native (confronto in aritmetica in complemento a 2)

<i>slt</i>	<i>rd</i> , <i>rs1</i> , <i>rs2</i>	if <i>rs1</i> < <i>rs2</i> then $rd := 1$ else $rd := 0$	confronto di <i>minoranza stretta</i>
<i>slti</i>	<i>rd</i> , <i>rs1</i> , <i>cost12</i>	if <i>rs1</i> < <i>cost12</i> then $rd := 1$ else $rd := 0$	confronto di <i>minoranza stretta</i> rispetto a <i>costante</i>

L'istruzione *slti* prima estende in segno la costante *cost12* da 12 bit fino a 64 bit, poi la confronta con il registro sorgente *rs1* e infine scrive il risultato nel registro destinazione *rd*. Le istruzioni native *sltu* e *sltiu* sono definite analogamente, ma il confronto è fatto in aritmetica naturale.

LOGICA – istruzioni native

<i>or</i>	<i>rd</i> , <i>rs1</i> , <i>rs2</i>	$rd := rs1 \text{ or } rs2$	<i>somma</i> logica bit a bit
<i>ori</i>	<i>rd</i> , <i>rs1</i> , <i>cost12</i>	$rd := rs1 \text{ or } cost12$	<i>somma</i> logica bit a bit con <i>costante</i> (est. in segno)
<i>and</i>	<i>rd</i> , <i>rs1</i> , <i>rs2</i>	$rd := rs1 \text{ and } rs2$	<i>prodotto</i> logico bit a bit
<i>andi</i>	<i>rd</i> , <i>rs1</i> , <i>cost12</i>	$rd := rs1 \text{ and } cost12$	<i>prodotto</i> logico bit a bit con <i>costante</i> (idem)
<i>xor</i>	<i>rd</i> , <i>rs1</i> , <i>rs2</i>	$rd := rs1 \text{ xor } rs2$	<i>somma</i> logica <i>esclusiva</i> bit a bit
<i>xori</i>	<i>rd</i> , <i>rs1</i> , <i>cost12</i>	$rd := rs1 \text{ xor } cost12$	<i>somma</i> logica <i>esclusiva</i> bit a bit con <i>cost.</i> (idem)

Le istruzioni *ori*, *andi* e *xori* prima estendono in segno la costante *cost12* da 12 bit fino a 64 bit, poi la compongono bit a bit con il registro sorgente *rs1* e infine scrivono il risultato nel registro destinazione *rd*.

LOGICA – pseudo-istruzioni

<i>not</i>	<i>rd</i> , <i>rs1</i>	$rd := \text{not } rs1$	<i>negazione</i> logica bit a bit (complemento bit a bit)
------------	------------------------	-------------------------	---

SCORRIMENTO – istruzioni native (costante intera positiva compresa nell'intervallo da 1 a 63)

<i>slli</i>	<i>rd</i> , <i>rs1</i> , <i>cost12</i>	$rd := rs1 \ll cost12$	scorrimento <i>logico verso sx</i> del numero di posizioni specificato da costante (introduce bit 0 da <i>dx</i>)
<i>srli</i>	<i>rd</i> , <i>rs1</i> , <i>cost12</i>	$rd := rs1 \gg cost12$	scorrimento <i>logico verso dx</i> del numero di posizioni specificato da costante (introduce bit 0 da <i>sx</i>)
<i>srai</i>	<i>rd</i> , <i>rs1</i> , <i>cost12</i>	$rd := rs1 \gg cost12$	scorrimento <i>aritmetico verso dx</i> del numero di posizioni specificato da costante (estende il segno)

Il terzo argomento, che dà il numero di posizioni da fare scorrere, può essere contenuto in un registro. Per esempio, l'istruzione nativa *sll rd, rs1, rs2* lo specifica nel registro *rs2*; idem per le istruzioni native *srl* e *sra*.

SALTO INCONDIZIONATO – istruzioni native – vedi nota 2

<i>j</i>	rd, spi20	rd := pc + 4 pc := pc + spi20	salto <i>incondizionato</i> relativo a PC con <i>salvataggio</i> dell'indirizzo di rientro nel registro <i>rd</i>
<i>jlr</i>	rd, spi12(rs1)	rd := pc + 4 pc := rs1 + spi12	<i>chiamata a sottoprogramma</i> relativa a registro <i>rs1</i> con <i>salvataggio</i> dell'indirizzo di rientro nel registro <i>rd</i>

SALTO INCONDIZIONATO – pseudo-istruzioni – vedi nota 2

<i>j</i>	spi20	pc := pc + spi20	salto <i>incondizionato</i> – salto relativo a PC
<i>jr</i>	rs1	pc := rs1	salto <i>incondizionato a lunga distanza</i> – salto da registro <i>rs1</i>
<i>jal</i>	spi20	ra := pc + 4 pc := pc + spi20	<i>chiamata a sottoprogramma</i> – salto incondizionato relativo a PC con <i>salvataggio</i> dell'indirizzo di rientro nel registro <i>ra</i>
<i>jlr</i>	rs1	ra := pc + 4 pc := rs1	<i>chiamata a sottoprogramma a lunga distanza</i> – salto incondizionato da registro <i>rs1</i> con <i>salvataggio</i> dell'indirizzo di rientro nel registro <i>ra</i>
<i>ret</i>		pc := ra	<i>rientro da sottoprogramma</i> – salto da registro <i>ra</i>

SALTO CONDIZIONATO – istruzioni native (confronto in aritmetica in compl. a 2) – vedi nota

<i>beq</i>	rs1, rs2, spi12	if rs1 = rs2 pc := pc + spi13	salta (relativamente a PC) se <i>uguale</i>
<i>bne</i>	rs1, rs2, spi12	if rs1 ≠ rs2 pc := pc + spi13	salta (idem) se <i>diverso</i>
<i>bge</i>	rs1, rs2, spi12	if rs1 ≥ rs2 pc := pc + spi13	salta (idem) se <i>maggiore o uguale</i>
<i>blt</i>	rs1, rs2, spi12	if rs1 < rs2 pc := pc + spi13	salta (idem) se <i>strettamente minore</i>

Le istruzioni native *bgeu* e *bltu* sono definite analogamente, ma il confronto è fatto in aritmetica naturale.

SALTO CONDIZIONATO – pseudo-istruzioni (confronto in aritmetica in compl. a 2) – vedi nota 2

<i>bgt</i>	rs1, rs2, spi12	if rs1 > rs2 pc := pc + spi13	salta (idem) se <i>strettamente maggiore</i>
<i>ble</i>	rs1, rs2, spi12	if rs1 ≤ rs2 pc := pc + spi13	salta (idem) se <i>minore o uguale</i>

Le pseudo-istruzioni *bgtu* e *bleu* sono definite analogamente, ma il confronto è fatto in aritmetica naturale.

Nota 2: Lo spiazzamento *spi12* o *spi20* (numero in complemento a 2) esprime la distanza di salto (positiva o negativa) in termini di mezze parole (half). L'indirizzo destinazione di salto, che verrà inviato alla memoria durante l'esecuzione del salto per prelevare l'istruzione collocata alla destinazione del salto, fa riferimento al byte. Durante l'esecuzione il processore moltiplica per 2 lo spiazzamento e così lo riferisce a byte. L'istruzione *jlr* (che è in formato I) fa eccezione: il processore *non* moltiplica per 2 lo spiazzamento *spi12*, ma si limita a *sovrascrivere* con 0 il bit meno significativo dello spiazzamento (se era 1 diventa 0, o resta 0).

TRASFERIMENTO DA / A MEMORIA – istruzioni native

<i>ld</i>	rd, spi12 (rs1)	rd := mem [rs1 + spi12]	carica <i>parola doppia</i> (da 64 bit)
<i>sd</i>	rs2, spi12 (rs1)	mem [rs1 + spi12] := rs2	memorizza <i>parola doppia</i> (da 64 bit)
<i>lw, lwu</i>	rd, spi12 (rs1)	rd := mem [rs1 + spi12]	carica <i>parola</i> (da 32 bit)
<i>sw</i>	rs2, spi12 (rs1)	mem [rs1 + spi12] := rs2	memorizza <i>parola</i> (da 32 bit)
<i>lh, lhu</i>	rd, spi12 (rs1)	rd := mem [rs1 + spi12]	carica <i>mezza parola</i> (da 16 bit)
<i>sh</i>	rs2, spi12 (rs1)	mem [rs1 + spi12] := rs2	memorizza <i>mezza parola</i> (da 16 bit)
<i>lb, lbu</i>	rd, spi12 (rs1)	rd := mem [rs1 + spi12]	carica <i>byte</i> (da 8 bit)
<i>sb</i>	rs2, spi12 (rs1)	mem [rs1 + spi12] := rs2	memorizza <i>byte</i> (da 8 bit)

Le istruzioni di trasferimento tra registri e memoria vanno usate con indirizzamento di memoria allineato, secondo la rispettiva dimensione di parola trasferita (*d*, *w*, *h*, *b*). Lo spiazzamento (in compl. a 2) è espresso come numero di byte. Solitamente esso è specificato come numero o come costante simbolica dichiarata tramite la direttiva *.eqv*. La scrittura *ld rd, (rs1)* sottintende spiazzamento nullo (similmente per tutte le altre *load* e tutte le *store*). Le istruzioni *lw*, *lh* e *lb* estendono in segno la parte superiore (non interessata dal caricamento) del registro destinazione *rd*, mentre le istruzioni *lwu*, *lhu* e *lbu* la azzerano.

TRASFERIMENTO TRA REGISTRI – pseudo-istruzioni

<i>mv</i>	rd, rs1	rd := rs1	<i>copia registro</i>
-----------	---------	-----------	-----------------------

CARICAMENTO DI COSTANTE E INDIRIZZO IN REGISTRO – istruzioni native

<i>lui</i>	rd, cost20	rd := cost20 (solo alcuni bit)	<i>caricamento (parziale) di costante</i> – carica la costante <i>cost20</i> nei 20 bit (5 cifre hex) più significativi della metà inferiore da 32 bit (8 cifre hex) del registro <i>rd</i> , azzera i 12 bit (3 cifre hex) meno significativi di <i>rd</i> ed estende in segno la metà inferiore di <i>rd</i> fino al 64-esimo bit
<i>auipc</i>	rd, spi20	rd := pc + spi20 (solo alcuni bit)	<i>caricamento (parziale) di indirizzo relativo a PC</i> – prima somma lo spiazzamento <i>spi20</i> all'indirizzo dell'istruzione <i>auipc</i> (contenuto nel registro <i>pc</i>), poi lo tratta come la costante nell'istruzione <i>lui</i> e lo carica nel registro <i>rd</i>

Solitamente la costante *cost20* è specificata come valore o come costante simbolica dichiarata tramite la direttiva *.eqv*. Lo spiazzamento *spi20* esprime la distanza (in byte) e di solito è specificato come etichetta di istruzione o di dato. Le istruzioni *lui* e *auipc* sono usate per espandere le pseudo-istruzioni *li* e *la*.

CARICAMENTO DI COSTANTE E INDIRIZZO IN REGISTRO – pseudo-istruzioni

<i>li</i>	rd, cost32	rd := cost32	<i>caricamento (completo) di costante</i>
<i>la</i>	rd, spi32	rd := pc + spi32	<i>caricamento (completo) di indirizzo relativo a PC</i>

Entrambe le pseudo-istruzioni *li* e *la* prima caricano un numero da 32 bit (word – 8 cifre hex), *cost32* e *spi32* rispettivamente, nella metà inferiore del registro destinazione *rd*, e poi estendono il segno della metà inferiore di *rd* alla metà superiore di *rd*. La pseudo-istruzione *li* (*load immediate*) carica una costante, di solito specificata come valore o come costante simbolica dichiarata tramite direttiva *.eqv*. La pseudo-istruzione *la* (*load address*) carica un indirizzo relativo a PC, di solito specificato come etichetta di istruzione macchina o di direttiva di dichiarazione di dato. Per vedere le espansioni di *li* e *la*, e come viene trattato e suddiviso l'eventuale simbolo che esprime l'argomento *cost32* e *spi32*, si legga di seguito.

COME VENGONO ESPANSE LE PSEUDO-ISTRUZIONI *LI* e *LA*

L'argomento della pseudo-istruzione *li* / *la* può essere un valore / un indirizzo da 32 bit già in forma numerica, oppure una costante simbolica / un'etichetta di istruzione o di dato. Se esso è una costante simbolica / un'etichetta, viene risolto nel corrispondente valore / indirizzo, e diventa un numero da 32 bit:

pseudo-istruzione	risoluzione dell'argomento
li rd, costante simbolica o valore	li rd, valore // num da 32 bit
la rd, etichetta o indirizzo	la rd, indirizzo // num da 32 bit

Inoltre per la pseudo-istruzione *la* (ma non per *li*) l'indirizzo viene reso relativo a PC, cioè viene convertito in distanza (*delta*) rispetto al registro *program counter* corrente (il quale contiene l'indirizzo dove è collocata *la*). La pseudo-istruzione *li* / *la* viene espansa nelle istruzioni native *lui* / *auipc* e (per entrambe) *addi*, suddividendo il numero da 32 bit in due componenti: 20 bit superiori (*hi*) e 12 bit inferiori (*lo*). Per rendere tale numero relativo a PC (*pcrel*) e suddividerlo, l'assemblatore utilizza questi modificatori:

modificatore	calcolo del modificatore - da parte del collegatore
% hi (val)	= val[31:12] + val[11] // 20 bit sup corretti
% lo (val)	= val[11:0] // 12 bit inf
si ponga: delta = ind - pc // l'indirizzo viene relativizzato a PC	
% pcrel_hi (ind)	= delta[31:12] + delta[11] // 20 bit sup corretti
% pcrel_lo (ind)	= delta[11:0] // 12 bit inf

Qui *val* / *ind* indica un numero da 32 bit, cioè un valore / un indirizzo dato come numero oppure ricavato risolvendo una costante simbolica / un'etichetta di istruzione o dato, e *delta* indica una distanza di salto relativamente a PC. I bit di *val* / *delta* sono numerati da 0 (bit meno significativo) a 31 (bit più significativo). L'assemblatore applica automaticamente i modificatori espandendo le pseudo-istruzioni *li* / *la*.

Ecco l'espansione di *li* / *la* tramite le istruzioni native e i modificatori predetti, e sotto la motivazione:

pseudo-istruzione	espansione - in istruzioni native e modificatori
li rd, valore	lui rd, %hi(valore) // 20 bit sup - HI addi rd, rd, %lo(valore) // 12 bit inf - LO
la rd, indirizzo	auipc rd, %pcrel_hi(indir) // 20 bit sup - HI addi rd, rd, %pcrel_lo(indir) // 12 bit inf - LO

L'istruzione *addi* prima estende in segno la costante da 12 bit fino a 64 bit e poi la somma al registro *rd*. Se il bit più significativo della costante vale 1, cioè se si ha *valore / delta* [11] = 1, dopo l'estensione la costante avrà assunto valore negativo. Poiché la costante esprime il componente da 12 bit, che in origine è positivo o nullo, occorre correggerla. Sommando il bit *valore / delta* [11] al componente da 20 bit, si corregge l'effetto dell'estensione di segno. L'assemblatore, generando il modulo oggetto, espande la pseudo-istruzione *li* / *la* tramite i modificatori. Il collegatore, unendo i moduli oggetto e generando l'eseguibile, calcola i modificatori e produce i componenti corretti da 20 bit e 12 bit che figurano in *lui* / *auipc* e *addi*.

COME VENGONO ESPANSE LE PSEUDO-ISTRUZIONI *NOP NEG NOT MV e BGT BLE*

pseudo-istruzione	espansione - in istruzione nativa
nop	addi zero, zero, 0 // reg zero è imm modificabile
neg rd, rs1	sub rd, zero, rs1 // rd = zero - rs1 = -rs1
not rd, rs1	xori rd, rs1, -1 // rd = rs1 xor 1 ⁶⁴ = not rs1
mv rd, rs1	addi rd, rs1, 0 // rd = rs1 + 0 = rs1
bgt rs1, rs2, spi	blt rs2, rs1, spi // scambia rs1>rs2 con rs2<rs1
ble rs1, rs2, spi	bge rs2, rs1, spi // scambia rs1<=rs2 con rs2>=rs1

COME VENGONO ESPANSE LE PSEUDO-ISTRUZIONI DI SALTO INCOND. *J JR JAL JALR e RET*

pseudo-istruzione	espansione - in istr. nativa jal(r) rd, spi(rs1)
j spi	jal zero, spi // non salva ind. di rientro
jr rs1	jalr zero, 0(rs1) // non salva ind. di rientro
jal spi	jal ra, spi // salva in ra ind. di rientro
jalr rs1	jalr ra, 0(rs1) // salva in ra ind. di rientro
ret	jalr zero, 0(ra) // prende da ra ind. di rientro

L'istruzione nativa *jal* effettua un salto con spiazzamento *spi* e salva l'indirizzo di rientro (*pc* + 4) nel registro destinazione *rd*. L'istruzione nativa *jalr* opera similmente, ma salta a registro base (registro sorgente) *rs1* più spiazzamento *spi*. Rimappando come sopra gli argomenti di queste due istruzioni native si ottengono tutte le pseudo-istruzioni elencate (*rd* = zero ha come effetto quello di non salvare l'indirizzo di rientro).

COME INDICARE I COMPONENTI DI UN'ISTRUZIONE MACCHINA E DI UN IMMEDIATO

Le istruzioni macchina di ISA RV64i sono codificate come parole da 32 bit. I bit dell'istruzione *inst* sono numerati da 0 a 31 (inclusi) e si indicano individualmente con *inst*[*k*] (con 31 ≥ *k* ≥ 0). La notazione *inst*[*k:h*] (con 31 ≥ *k* > *h* ≥ 0) indica il campo di *k* - *h* + 1 ≥ 2 bit compreso tra i bit *k*^{esimo} e *h*^{esimo} (inclusi) di *inst*. Similmente un immediato *imm* (costante o spiazzamento) ha varia lunghezza (secondo il formato dell'istruzione cui appartiene) e i suoi bit e campi di bit si indicano con *imm*[*k*] e *imm*[*k:h*], rispettivamente.

istruzione macchina nativa *inst* (ISA RV64i) – i campi di bit da 31 a 7 (inclusi) dipendono dal formato

31

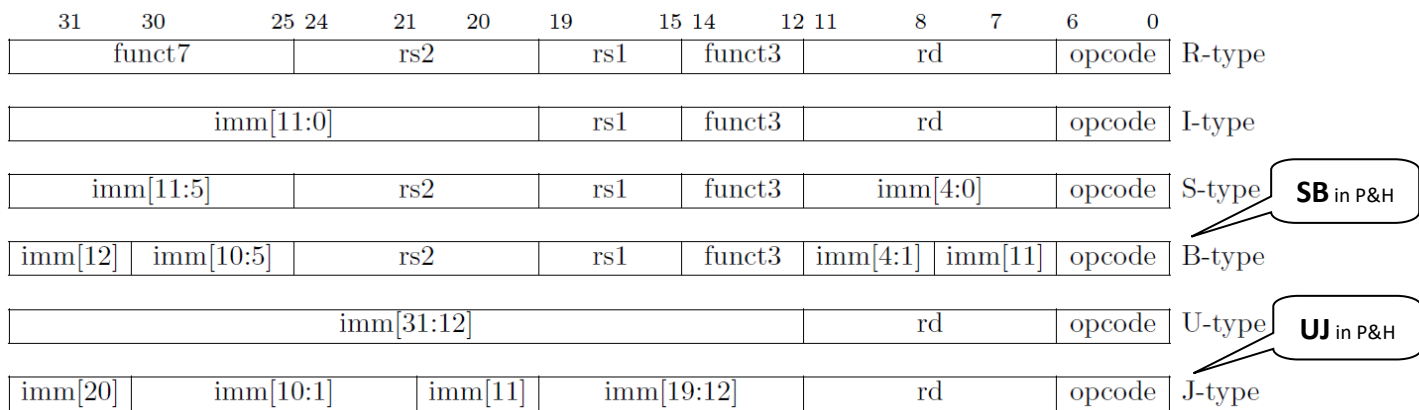
7 6

0

immediato (<i>imm</i>) / indirizzi di registro (<i>rd, rs1, rs2</i>) codici operativi ausiliari (<i>funct</i>)	codice operativo (<i>opcode</i>)
---	---------------------------------------

Il campo *inst*[6:0] è di 7 bit e codifica sempre il codice operativo dell'istruzione *inst*. I bit rimanenti [31:7] sono suddivisi in vari campi e codificano un eventuale immediato, gli indirizzi dei registri dati come argomenti, ed eventuali codici operativi ausiliari (*funct7* e *funct3*), secondo il formato dell'istruzione *inst*.

FORMATI NUMERICI A 32 BIT DELLE ISTRUZIONI MACCHINA NATIVE – vedi nota 3

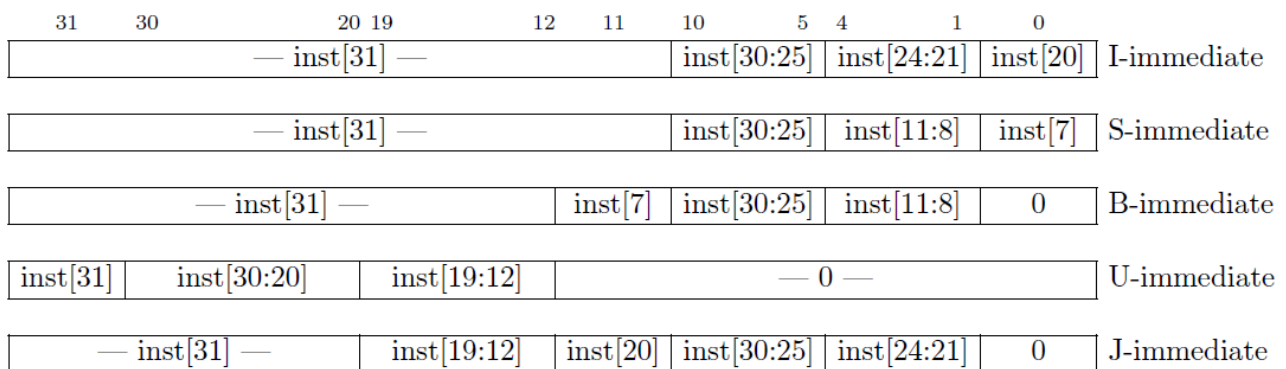


Nel libro di **Patterson & Hennessy** i formati *B* e *J* sono chiamati **SB** e **UJ**, per la loro somiglianza con *S* e *U*.

Nota 3: Il formato *B* (con due registri sorgente) contiene uno spiazzamento *imm*[12:1] da 12 bit, che esprime la distanza di salto (positiva o negativa) in termini di mezze parole (*half*). Il processore *moltiplica* lo spiazzamento per 2 (*aggiungendogli* un bit 0 in coda), così *referendolo a byte*. Questa è l'unica differenza tra i formati *S* e *B*. Similmente per i formati *U* e *J*, con spiazzamento da 20 bit (con un registro destinazione).

COSTRUIRE UN IMMEDIATO DAI BIT DELL'ISTRUZIONE ED ESTENDERLO IN SEGNO FINO A 32 BIT

Durante l'esecuzione, il processore RISC-V costruisce l'immediato (costante o spiazzamento) a 32 bit a partire dai bit *imm* contenuti nell'istruzione *inst*, secondo questi schemi dipendenti dal formato di *inst*.



L'estensione in segno a 32 bit di un immediato (laddove è prevista) viene ottenuta replicando il bit *inst*[31] dell'istruzione e qui è indicata con — *inst*[31] —. La notazione — 0 — indica una sequenza di bit 0 *aggiunti* in coda. Dopo essere stato costruito a 32 bit, l'immediato può essere ancora esteso in segno fino a 64 bit.

CORRISPONDENZA TRA FORMATI NUMERICI E ISTRUZIONI MACCHINA NATIVE – vedi nota 4

formato	istruzione	aggiunte e varianti
<i>R</i>	<i>add, sub, or, and, xor, slt</i>	<i>mul, div, rem, sll, srl, sra, sltu</i>
<i>I</i>	<i>addi, slti, ori, andi, xori, slli, srli, srai, ld, lw, lh, lb, jalr</i>	<i>sltiu, lwu, lhu, lbu</i>
<i>S</i>	<i>sd, sw, sh, sb</i>	
<i>B</i> (o <i>SB</i>)	<i>beq, bne, bge, blt</i>	<i>bgeu, bltu</i>
<i>U</i>	<i>lui, auipc</i>	
<i>J</i> (o <i>UJ</i>)	<i>jal</i>	

Nota 4: L'istruzione *jalr*, pur effettuando un salto a istruzione, ha formato *I* (non *J*) e il processore non ne allinea lo spiazzamento *spi12* a mezza parola (*half*) aggiungendo in coda il bit 0, come fa per le istruzioni in formato *B* e *J* (nota 2). Invece, per *jalr* il processore *azzerà* il bit meno significativo *imm*[0] dello spiazzamento, benché esso figuri nel formato *I* e quivi possa valere 1. Dunque per *jalr* i bit liberi di *spi12* sono solo 11 e la max distanza di salto è la metà di quella per i salti condizionati *bxx* e per l'istruzione *jal*.

REGISTRI REFERENZIABILI – nominabili come argomento di istruzione macchina

#	nome ABI	utilizzo
0	<i>zero</i>	costante 0 – imm modificabile (anche se viene scritta non cambia valore)
1	<i>ra</i>	return address – indirizzo di rientro da funzione
2	<i>sp</i>	stack pointer – puntatore alla cima della pila
3	<i>gp</i>	global pointer – puntatore all'area dati globale QUI NON USATO
4	<i>tp</i>	thread pointer – puntatore a thread QUI NON USATO
5 – 7	<i>t0 – t2</i>	valori temporanei (indirizzi, variabili globali e calcolo delle espressioni)
8	<i>s0 / fp</i>	variabile locale di funzione / frame pointer (se in uso)
9	<i>s1</i>	variabile locale di funzione
10 – 11	<i>a0 – a1</i>	valore restituito da funzione (di solito basta <i>a0</i> – se serve si aggiunge <i>a1</i>)
12 – 17	<i>a2 – a7</i>	argomenti in ingresso a funzione (max 6 argomenti, gli eccedenti in pila)
18 – 27	<i>s2 – s11</i>	variabili locali di funzione (max 10 o 11 variabili, le eccedenti in pila)
28 – 31	<i>t3 – t6</i>	valori temporanei (indirizzi, variabili globali e calcolo delle espressioni)

Nelle (pseudo-)istruzioni macchina i registri vanno indicati di preferenza tramite i loro nomi ABI, perché il nome ABI esprime l'uso del registro. I due usi previsti per il registro *x8* (*s0* oppure *fp*) sono mutuamente esclusivi. Se il frame pointer *fp* non è in uso, il registro *s0* è disponibile per allocare una variabile locale di funzione, altrimenti per allocare si inizia dal registro *s1*. I registri *gp* e *tp* sono previsti per compatibilità con altri modelli di processore (p. es. MIPS) e per programmazione concorrente, ma qui non sono usati.

SALVATAGGIO E RIPRISTINO DEI REGISTRI – salvare / ripristinare i registri in / da pila a 64 bit

ruolo	registro/i	dettagli
caller-saved	<i>t0 – t6, a0 – a7</i>	salvare in / ripristinare da pila (se serve) nell'ordine indicato
callee-saved	<i>fp, ra, s0 – s11</i>	salvare in / ripristinare da pila (se serve) nell'ordine indicato

Il registro frame pointer *fp* va salvato e ripristinato solo se è in uso (nel qual caso il registro *s0* non esiste).

DIRETTIVE FONDAMENTALI DI ASSEMBLATORE RISC-V – per RARS e GCC (con eccezioni)

etichetta	direttiva	argomento/i	funzione della direttiva
no	.align	0 o 1 o 2 o 3	allinea il dato successivo a <i>byte</i> o <i>half</i> o <i>word</i> o <i>dword</i>
sì	.ascii(z)	"stringa"	metti costante <i>stringa</i> in memoria (con NULL in fondo)
sì	.byte	<i>c1, c2, ...</i>	metti il/i byte con valore iniziale <i>c1, c2, ...</i> in memoria
no	.data	<i>ind</i> a 64 bit	dichiara segmento dati a <i>ind</i> (<i>ind</i> è opzionale)
sì	.dword	<i>d1, d2, ...</i>	metti parola/e doppia/e (double) con v.i. <i>d1, d2, ...</i> in mem
no	.eqv	<i>sym, val</i>	dichiara costante simbolica <i>sym</i> con valore <i>val</i> (#define)
no	.globl	<i>sym</i>	esporta simbolo <i>sym</i> ad altri moduli oggetto
sì	.half	<i>h1, h2, ...</i>	metti mezza/e parola/e (half) con v.i. <i>h1, h2, ...</i> in mem
sì	.space	<i>num</i>	alloca <i>num</i> byte di spazio non inizializzato – RARS
no	.text	<i>ind</i> a 64 bit	dichiara segmento testo (codice) a <i>ind</i> (<i>ind</i> è opzionale)
sì	.word	<i>w1, w2, ...</i>	metti parola/e (word) con v.i. <i>w1, w2, ...</i> in memoria
sì	.zero	<i>num</i>	alloca <i>num</i> byte di spazio BSS (inizializzato a 0) – GCC

L'allineamento di default è **.align 0**, cioè allineamento a byte ossia nessun allineamento: in questo caso le direttive di dichiarazione di dato **.ascii(z)**, **.byte**, **.dword**, **.half**, **.space**, **.word** e **.zero** allocano lo spazio di memoria richiesto a partire dal primo byte libero. Si suppone che il processore e la memoria, a livello HW, supportino una tale modalità di accesso non allineata (processori/memoria diversi possono differire).

Le direttive **.space** e **.zero** sono molto simili. La seconda (riconosciuta da GCC) alloca spazio di memoria BSS (Linux), che il sistema operativo Linux inizializza a 0 quando carica il processo in memoria.

Altre direttive (qui non usate): **.macro** ed **.end_macro** per dichiarare una macro (pseudo-istruzione), e **.rodata** (GCC) per dichiarare un segmento di dati non modificabili (*read-only data*). La direttiva **.rodata** serve principalmente per dichiarare tabelle globali fisse di costanti lunghe o di puntatori / spiazziamenti (*GOT – Global Offset Table*) a variabili a lunga distanza, o per strutture dati complesse imm modificabili.